

Agent Runtime 技术全景

Agent Runtime 技术调研 – 工具集成与执行 (Tool)

子领域: Agent 如何与外部世界交互 – 浏览器操作、网页内容提取、SaaS API 调用

一、核心问题

Agent 的智能不仅来自 LLM 推理，更来自与外部世界的交互能力。工具集成与执行子领域回答一个根本问题：**Agent 如何获得「手」和「眼」？**

1.1 「手」的问题：Agent 如何操作网页？

LLM 天然只能处理文本，无法点击按钮、填写表单、滚动页面。browser-use 的思路是：**给 Agent 一个真实的浏览器，把 DOM 结构和页面截图同时喂给 LLM，让 LLM 输出下一步操作指令**。这是一种「视觉+结构」双通道理解模式。

但这里有一个关键张力：**人类看网页不需要 DOM，为什么 Agent 需要？** 实际上，DOM 提供的是精确的元素定位坐标（CSS selector / index），视觉提供的是语义理解（「找到那个蓝色按钮」）。两者缺一不可：

- **纯视觉方案**（截图→VLM→坐标）：直观但精度差，受分辨率、缩放、动态布局影响
- **纯 DOM 方案**（HTML→LLM→selector）：精确但 token 消耗大，动态生成的 class/id 难以定位
- **混合方案**（browser-use 的做法）：DOM 提供可操作元素列表（索引标签），截图提供视觉上下文 – LLM 输出索引号而非坐标

1.2 「眼」的问题：Agent 如何读懂网页？

操作网页之前，Agent 先要理解网页上有什么。这是内容提取层解决的问题：

- **firecrawl** 走的是**深度 route**: JS 渲染 → 智能去噪（去导航栏/广告/页脚）→ Markdown 输出 → 可选结构化 JSON。适合「把这个网站的所有文档爬下来做 RAG」。
- **reader** 走的是**轻量 route**: URL 前缀 `r.jina.ai` → 自动识别格式 → 返回 Markdown。适合「Agent 临时需要读这个网页」的即时场景。
- **page-agent** 走的是**语义 route**: DOM → 简化语义树 → LLM 友好的结构化表示。适合「Agent 需要理解复杂的页面结构和关系」。

三者的差异本质上是**抓取深度**和**使用复杂度**的权衡：

	firecrawl	reader	page-agent
使用门槛	API Key + 配置	URL 前缀即可	需集成代码
抓取深度	整站点递归爬取	单页即时转换	单页语义分析
输出格式	Markdown / JSON / Screenshot	Markdown / HTML / Screenshot	语义化 JSON
反反爬能力	代理池 + UA + Cookie 管理	API Key 信任池 + 代理 + 浏览器引擎	不涉及
自托管	Docker 部署	Docker 单镜像	Python 库

1.3 「身份」的问题：Agent 如何安全调用 SaaS API?

让 Agent 帮用户发邮件、查日历、读 GitHub PR，首先面临的是**认证问题**。这不是技术难度最高的环节，但是**用户体验的最大瓶颈**：

- 用户需要去每个 SaaS 平台生成 API Key
- 每个 API Key 的权限粒度和有效期策略不同
- Token 过期后 Agent 静默失败，用户体验极差
- 用户无法精细控制「Agent 可以读我的邮件但不能发」

composio 的方案是：把认证变成**托管服务**。用户只需在 Composio 中授权一次（OAuth），后续所有 Agent 框架通过 Composio 的统一 API 调用工具，Composio 负责 Token 刷新、权限检查和审计日志。这在架构上把「认证」从 Agent 框架中解耦出来，成为独立的中间层。

1.4 底层 vs. 上层：浏览器控制的抽象层级

browser-use 生态内有一个重要的分层设计：

browser-use (Agent 决策层)
└─ browser-harness (浏览器控制框架)

- **Playwright**: 最底层, 提供 `page.click(selector)`、`page.type(selector, text)` 等原子操作
- **browser-harness**: 中层抽象, 封装浏览器启动配置、代理、Cookie、Tab 管理等
- **browser-use**: 上层 Agent, 将 LLM 的自然语言意图翻译为对 browser-harness 的操作调用

这个分层让开发者可以**按需选择抽象层级**: 不需要 Agent 时直接用 browser-harness 做纯自动化, 需要 Agent 时才引入 browser-use。

二、技术方向

方向 1: 浏览器自动化 (Browser Automation)

让 Agent 像人类一样操作浏览器。

代表项目: browser-use、browser-harness、agent-browser、page-agent

核心技术挑战:

1. **元素定位**: 网页每天都在变, CSS class 是动态生成的, XPath 脆弱。browser-use 的解决方案是**索引标签注入** – 在每个可操作元素旁边注入一个数字标签, LLM 只需输出数字即可定位元素。这是一个工程化技巧, 但有效解决了 LLM 输出 selector 不稳定的问题。
2. **多步骤规划与执行**: 「查找 100 美元以下的蓝牙耳机」需要分解为「搜索→浏览结果→点击→看价格→比较→决策」多步。browser-use 通过 LLM 的 ReAct 循环实现: 每一步观察当前页面状态 → 决定下一步操作 → 执行 → 观察新状态。
3. **视觉 vs. DOM 的互补**: 当 DOM 结构混乱 (如 Canvas 渲染的页面) 时, 纯 DOM 定位失效, 必须依靠截图 + VLM 的视觉定位。browser-use 的「DOM + 视觉双模式」是这个方向的标杆方案。
4. **反检测 (Stealth)**: 网站越来越擅长识别和拦截自动化浏览器。这不仅是 Puppeteer/Playwright 的 `--no-sandbox` 参数问题, 还包括 WebDriver 标记、浏览器指纹、鼠标轨迹模拟等多个维度。browser-use 内置了 Stealth 模式, 但这本质上是一场**持续的军备竞赛**。
5. **轻量 vs. 重量**: agent-browser 代表了另一种哲学 – 不要试图覆盖所有浏览器操作, 只提供 Agent 最常用的 4 个操作 (`goto/click/screenshot/extract`), API 极简, 可以在 Vercel

Serverless (10s 超时) 环境中运行。这适合**快速原型和简单任务**，但对于需要登录、多页面跳转的复杂场景则力不从心。

方向 2：网页内容提取 (Web Content Extraction)

将任意网页转化为 LLM 可消费的格式。

代表项目：firecrawl、reader

核心技术挑战：

1. **JS 渲染 vs. 静态抓取**：SPA (React/Vue/Angular) 页面需要无头浏览器渲染，但速度慢、资源消耗大。firecrawl 和 reader 都实现了双引擎 – 对静态页面走轻量 HTTP 请求，对 SPA 自动切换到浏览器渲染。reader 的 `curl-impersonate` 方案更进一步，模拟浏览器 TLS 指纹来绕过基本的 CDN 防护。
2. **内容去噪 (Content Extraction)**：网页中 80% 的内容是导航栏、侧边栏、广告、页脚、推荐链接。核心挑战是从 HTML 噪声中提取「正文」。firecrawl 使用 ML 模型做内容识别，reader 使用启发式规则 + CSS 选择器。这是一个**看起来简单但工程上极其棘手**的问题 – 每个网站的 HTML 结构都不同。
3. **深度爬取 (Crawling)**：从单个 URL 出发，自动发现并抓取同域下所有链接页面。需要处理的问题包括：避免死循环 (URL 参数变化但内容相同)、速率控制 (不能打垮目标站点)、去重策略、增量更新。
4. **输出格式优化**：目标是让 LLM 高效消费。Markdown 是共识输出格式 (保留标题层级、列表、表格) – firecrawl 和 reader 都用 Markdown 作为默认输出。firecrawl 还支持结构化 JSON 提取 (如产品名称+价格+评分)，reader 支持语义切片 (按标题层级切分，直接适配 RAG 的 chunking)。

方向 3：工具集成平台 (Tool Integration Platform)

将 Agent 的工具调用标准化，尤其是认证和权限管理。

代表项目：composio

核心技术挑战：

1. **Managed Auth (托管认证)**：这是 composio 的核心价值。普通工具集成要求开发者处理 OAuth2 流程、Token 存储和刷新、权限范围管理。Composio 把这些全部托管 – Agent 框架只需调用 `composio.tool(gmail, action='read_emails')`，Composio 自动处理底层认证。这大大降低了 Agent 工具集成的门槛。

2. **Action 级别权限控制**：不是「Agent 可以访问 Gmail」，而是「Agent 可以读邮件但不能发邮件」。这种细粒度控制对**企业场景**至关重要 – 没有人愿意给 Agent 完整的 SaaS 账号权限。
3. **多框架兼容**：Agent 框架生态碎片化严重（LangChain / CrewAI / AutoGen / OpenAI SDK / Anthropic SDK 各不兼容）。Composio 的策略是**不做框架，做工具层** – 为所有主流框架提供适配器，让框架之间共享同一套工具和认证配置。
4. **事件驱动工具调用**：Agent 不只是在用户请求时被动调用工具，还可以订阅外部事件（新邮件到达、新 Issue 创建）主动触发行动。Composio 的 Trigger 机制让 Agent 从「请求-响应」模式扩展到「事件驱动」模式。

三、趋势分析

趋势 1：从单一工具到工具组合

早期 Agent 项目通常只集成 1-2 个工具（如搜索+浏览器）。现在的趋势是**多工具组合**：一个 Agent 同时使用浏览器操作（browser-use）+ 内容提取（firecrawl/reader）+ SaaS API（composio）+ 代码执行（sandbox）。工具之间不是互斥的，而是**互补的**。

典型组合模式：

用户：「帮我调研竞品 X 的产品更新，汇总到 Notion，然后发邮件给团队」

Agent 执行流：

1. firecrawl 爬取竞品 X 官网和博客 → Markdown 内容
2. browser-use 操作竞品 X 的 Web App（需要登录和交互）→ 截图和操作结果
3. composio 调用 Notion API 创建汇总页面
4. composio 调用 Gmail API 发送通知邮件

趋势 2：轻量化和 Serverless 化

agent-browser 和 reader 代表了**轻量化**趋势：不是每个 Agent 都需要完整的 Playwright 浏览器。对于大量简单场景（读一篇文章、提取一个页面的关键信息），「URL 前缀即用」或「4 个核心 API」的模式比部署完整的浏览器集群更实用。

这与 Vercel / Cloudflare Workers / AWS Lambda 等 Serverless 平台的普及密切相关 – Serverless 函数有严格的执行时间和内存限制（通常 10s-30s），传统浏览器自动化无法适应。

趋势 3：内容提取的 LLM-Native 化

firecrawl 和 reader 的核心设计理念是**输出格式面向 LLM 优化**，而非面向人类阅读。Markdown 输出、语义切片、结构化 JSON 提取、LLM 友好的元数据（Frontmatter）– 这些都是在为 RAG、Fine-tuning、Agent 上下文构建做准备。

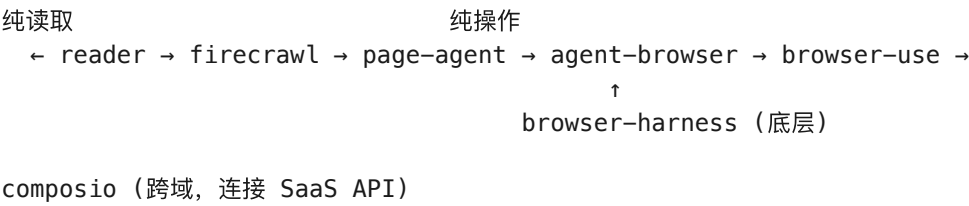
这与传统爬虫（输出 HTML / 数据库存储）形成了鲜明对比：传统爬虫的数据需要二次清洗才能喂给 LLM，而 firecrawl/reader 直接输出 LLM 可消费的数据。

趋势 4：认证即服务（Auth-as-a-Service）

composio 的方向代表了 Agent 工具集成的**标准化趋势**：认证和权限管理不应由每个 Agent 框架或每个开发者重复实现，而应该是一个**独立的托管服务**。这与「身份平台」（Auth0 / Okta / Clerk）在传统 Web 应用中的角色类似 – 认证从应用中解耦，成为基础设施。

趋势 5：交互深度的光谱

7 个项目形成了从「只读」到「完全交互」的完整光谱：



这反映了 Agent 工具生态的成熟：不同场景、不同复杂度、不同部署环境，有不同定位的工具可选。

趋势 6：反爬与反反爬的持续演进

凡是涉及网页操作和内容提取的项目，反检测都是核心能力。网站的反爬策略越来越复杂（Cloudflare Bot Management、DataDome、reCAPTCHA v3），而 Stealth 模式、代理轮换、浏览器指纹伪装等手段也在持续升级。**这是一个没有终点的技术竞赛。**

四、项目全景表

项目	组织	定位	核心能力	技术栈	代码规模	状态
browser-use	browser-use	Agent 浏览器操作	DOM+视觉双模式、多步规划、多 Tab、Stealth	Python, Playwright, LangChain	5.5K 节点	活跃

项目	组织	定位	核心能力	技术栈	代码规模	状态
browser-harness	browser-use	浏览器控制框架	Playwright 高级封装、可配置 Harness	Python, Playwright	-	活跃
agent-browser	Vercel Labs	轻量 Agent 浏览器	4 核心 API、Serverless 兼容、AI SDK 集成	TypeScript, Puppeteer	-	活跃
page-agent	Alibaba	网页语义理解	DOM 语义化转换、LLM 友好输出	Python, DOM Parser	-	孵化
firecrawl	firecrawl	AI 就绪网页抓取	JS 渲染+静态双模式、深度爬取、Markdown/JSON 输出、反反爬	TypeScript, Rust, Puppeteer/Playwright	13.6K 节点	活跃
reader	Jina AI	轻量内容提取	URL 前缀即用、多格式解析、搜索集成、预配置	TypeScript, Puppeteer, curl-impersonate	-	活跃
composio	ComposioHQ	工具集成平台	Managed Auth、200+ SaaS 工具、Action 权限、事件触发	TypeScript, Python, PostgreSQL	9.7K 节点	活跃

四类工具的分工关系

Agent 决策层（LLM）			
浏览器操作 （方向1）	内容提取 （方向2）	SaaS API （方向3）	代码/Shell （其他）
browser-use browser-harness agent-browser page-agent	firecrawl reader	composio	sandbox tools

竞合关系矩阵

	browser-use	agent-browser	page-agent	firecrawl	reader	composio
browser-use	-	竞品	互补/竞品	互补	互补	工具提供
agent-browser	竞品	-	无直接关系	无直接关系	无直接关系	无直接关系
firecrawl	互补	无直接关系	互补	-	竞品	无直接关系
reader	互补	无直接关系	无直接关系	竞品	-	无直接关系
composio	集成方	无直接关系	无直接关系	无直接关系	无直接关系	-

browser-use 与 agent-browser 是直接竞品（同一赛道、不同重量级）；firecrawl 与 reader 是直接竞品（内容提取赛道、不同侧重点）；composio 是平台方，browser-use 是其上的高频工具，而非竞品。

五、关键洞察

5.1 「手」和「眼」正在融合

browser-use 的 DOM+视觉双模式、page-agent 的语义化 DOM 理解、firecrawl 的智能内容提取 – 这些都在模糊「操作」和「理解」的界限。未来的 Agent 浏览器工具可能不再区分「我是先看还是先点」 – LLM 在每一步同时进行**页面理解**和**操作决策**，二者交替推进。

5.2 工具层的标准化是最大瓶颈

当前 Agent 工具生态最大的问题不是「工具不够多」，而是**每个框架的工具接口不兼容**。LangChain 的 Tool、OpenAI 的 Function Calling、Anthropic 的 Tool Use – 同一把 Gmail 工具需要为每个框架写一遍集成代码。composio 试图解决这个问题，但这需要**生态级共识**。

5.3 反检测能力是硬壁垒

浏览器自动化和网页抓取项目中，**反检测能力是核心竞争力**。一个能绕过 Cloudflare Bot Management 的 firecrawl / browser-use 实例，与一个裸奔的 Playwright 脚本，在生产环境中的可用性是天差地别的。这也是为什么自托管版本和云托管版本在反反爬能力上通常存在差距。

5.4 轻量方案不替代重量方案，但扩大了使用场景

agent-browser 无法替代 browser-use 做复杂的多步骤操作，reader 无法替代 firecrawl 做深度站点爬取。但轻量方案**降低了 Agent 工具使用的门槛** – 「URL 前缀即可获取页面内容」的设计让

更多简单场景无需引入重型浏览器基础设施就能实现。

5.5 安全与信任是隐含但关键的主题

composio 的 Managed Auth、Action 级别权限控制、审计日志 – 这些都是**企业级 Agent 部署**的必需能力。当 Agent 开始访问用户的 Gmail、GitHub、Salesforce，安全问题就不再是 nice-to-have，而是 must-have。这与「沙箱安全」子领域形成了呼应关系。

六、与其他 Agent Runtime 子领域的关系

- **沙箱 (sandbox)**: 浏览器操作天然需要沙箱环境 – browser-use 启动的浏览器实例就是一个沙箱化的执行环境，需要隔离、资源限制和安全策略
 - **网关 (gateway)**: 工具调用的路由和负载均衡可以由 API Gateway 处理，尤其是 composio 的多工具管理
 - **可观测性 (observability)**: 工具调用的成功/失败、延迟、Token 消耗需要纳入 Agent 的 tracing 体系
 - **协议 (protocol)**: 工具调用的标准化接口（如 A2A 的工具定义）与 composio 的工具集成方案有交叉
 - **安全 (security)**: 网页操作的内容安全（XSS 注入、恶意页面）和 API 调用的权限控制需要安全子领域的覆盖
-

七、开放问题

1. **CAPTCHA 突破**: 当前 browser-use / firecrawl 的反检测策略能否应对日益复杂的 CAPTCHA（如 reCAPTCHA v3 的行为分析）？是否有利用 VLM 视觉能力直接「看懂并点击」CAPTCHA 的方案？
2. **多工具编排的可靠性**: 当 Agent 同时使用 5+ 工具时，工具之间的状态一致性和错误恢复如何处理？一个工具调用失败是否应该回滚之前的操作？
3. **Managed Auth 的安全审计**: composio 托管了用户所有 SaaS Token，这在理论上是一个**极高价值的攻击目标**。其安全架构如何确保 Token 不泄漏？
4. **浏览器会话的持久化**: browser-use 的浏览器会话是否可以跨多次 Agent 调用保持？如果可以，Cookie/登录态的安全管理如何保证？
5. **成本优化**: 浏览器自动化每次操作都需要 LLM 推理（看页面→决策→操作），多步骤操作的成本可能指数级增长。是否有 caching / batching 的优化空间？

产出时间: 2026-07-07 | 数据来源: *projects/agent-runtime/tool//summary.md* + codebase-memory 架构数据*